

Klasyfikacja stopnia bólu u mysz laboratoryjnych

Rozpoznanie za pomocą YOLO

Oliwia Pawelec

10 czerwca 2025



Spis treści

1	Wprowadzenie	3
2	Zbiór danych	3
2.1	Eksploracyjna Analiza Danych	3
2.2	Augmentacja	4
3	Modele treningowe	5
3.1	Implementacja YOLO	6
3.1.1	Backbone (Szkielet)	6
3.1.2	Detection Head (Detekcja)	6
3.1.3	Opis implementacji	7
3.2	Klasyfikator bólu	7
3.3	Klasyfikator bólu	7
3.3.1	Modyfikacja architektury	7
3.3.2	Opis implementacji	8
3.4	Funkcja straty YOLO	8
3.4.1	Komponenty funkcji straty	8
3.4.2	Mechanizm anchorów	8
3.4.3	Opis implementacji	9
3.5	System zarządzania modelem	9

3.5.1	Architektura systemu	9
3.5.2	Pipeline przetwarzania	9
3.5.3	Zarządzanie danymi	9
3.5.4	Implementacja systemu	10
3.6	Wyniki treningu	10
3.6.1	Wyniki YOLO	10
3.6.2	Wyniki klasyfikatora bólu	10
3.7	Analiza wyników	11
3.7.1	Przykłady detekcji	12
4	Bibliografia	12

1 Wprowadzenie

Projekt indywidualny powstały w ramach toku studiów na Politechnice Warszawskiej. Dotyczy on rozpoznania stopnia bólu na twarzach myszy laboratoryjnych z wykorzystaniem skali MGS (Mouse Grimace Scale), opracowanej przez Langford i jej zespół (pod kierunkiem dr Jeffreya Mogila) na McGill University w 2010 roku. Jej założeniem jest, że zmiany w wyrazie twarzy, takie jak zwężenie oczu, zmiany położenia wąsów, wybrzuszenie nosa/policzków czy zmiana ustawienia uszu, stanowią wiarygodny i szybki sposób oceny bólu. Analogiczne skale zostały opracowane również dla innych zwierząt laboratoryjnych, takich jak szczury (RGS) i króliki (RbtGS).

W ramach projektu zaimplementowano system do automatycznej analizy obrazów twarzy myszy z wykorzystaniem sieci neuronowej opartej na architekturze YOLO (You Only Look Once), umożliwiającej jednoczesną detekcję kluczowych punktów twarzy (np. oczu, wibrysów, uszu) i klasyfikację obrazów zbliżoną do czasu rzeczywistego. System ten został wytrenowany na zbiorze danych zawierającym ponad 3000 zdjęć myszy z ręcznie oznaczonymi ocenami MGS, co pozwoliło na stworzenie modelu zdolnego do szybszej i bardziej obiektywnej oceny bólu u zwierząt laboratoryjnych w porównaniu do tradycyjnej, manualnej oceny przez człowieka.

Zastosowanie zautomatyzowanego podejścia może znacząco przyczynić się do poprawy dobrostanu zwierząt laboratoryjnych poprzez szybką identyfikację osobników odczuwających ból. Gryzonie maskują objawy bólu, dlatego analiza mikroekspresji twarzy może pomóc we wcześniejszym wykrywaniu stanów chorobowych, które często diagnozowane są dopiero w zaawansowanych stadiach, takich jak np. guz przysadki.

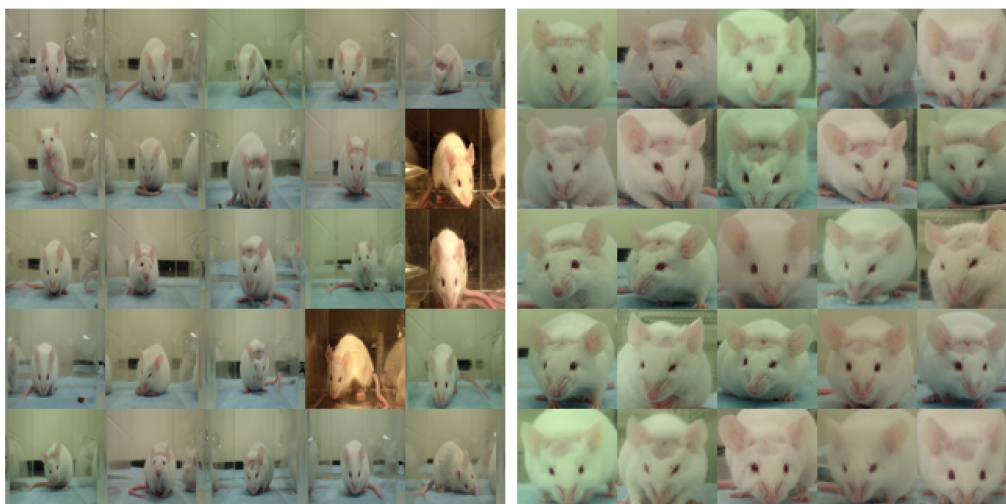
2 Zbiór danych

Dane zostały zgromadzone przez A. Vidal, S. Jha, S. Hassler, T. Price, C. Busso i wykorzystane pierwotnie w artykule "Face detection and Grimace Scale Prediction of White Furred Mice" z czasopisma MLWA z 2022 roku.

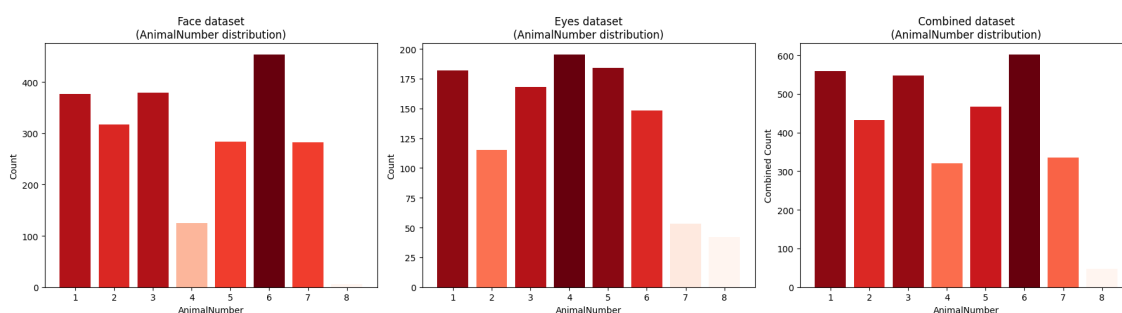
Zbiór, z którego skorzystałam, został umieszczony na stronie Multimodal Speech Processing (MSP) Laboratory: lab-msp.com/WhiteFurredMiceDataset/. Opis zdjęć ze zbioru (tzn. położenie bounding boxów do wykrywania ciała/pyszczka myszy) został wzięty ze strony github.com/AndreaVidal/WhiteFurredMice_Dataset/. Jest on również częścią wyżej wspomnianej pracy naukowej.

2.1 Eksploracyjna Analiza Danych

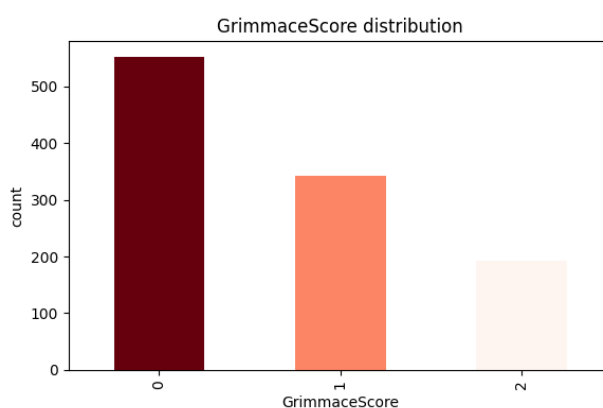
Przeprowadziłam eksploracyjną analizę danych. W zbiorze ze strony MSP Laboratory jest 3309 plików zdjęciowych, każde z nich z dokładnie wyznaczonymi bounding boxami. 2222 zawiera opis położenia ciała myszy, a pozostałe 1087 opis położenia pyszczka. Zdjęcia zawierają łącznie 8 różnych osobników gatunku myszy laboratoryjnej. Najwięcej osobników o numerze 6 (601), a najmniej o numerze 8 (48).



Rysunek 1: Przykładowe obrazy twarzy i oczu ze zbioru danych



Rysunek 2: Rozłożenie liczby różnych osobników w zbiorze



Rysunek 3: Rozłożenie wyniku ze skali grymasu w zbiorze

2.2 Augmentacja

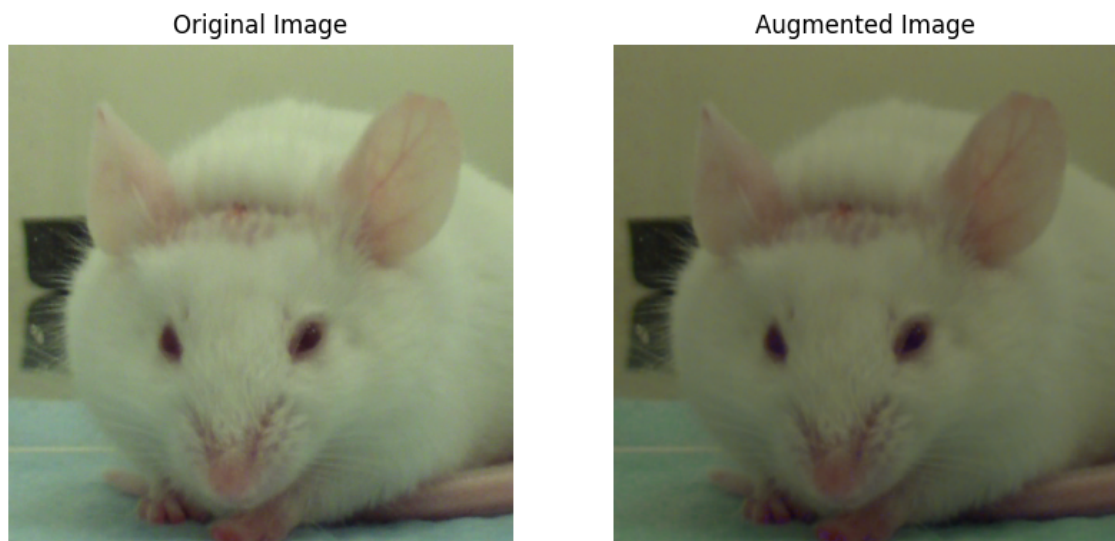
Zbiór 3000 zdjęć jest dość ubogi do treningu modelu YOLO, zatem by polepszyć dokładność w rozpoznawaniu twarzy i oczu na zdjęciach, potrzebna była augmentacja. Zastosowałam następujące czynniki:

- losowe odbicie poziome (50% szansy)

- losowe skalowanie (od 80% do 120% oryginalnego rozmiaru)
- zmiany kolorów w przestrzeni HSV (tzw. jitter).



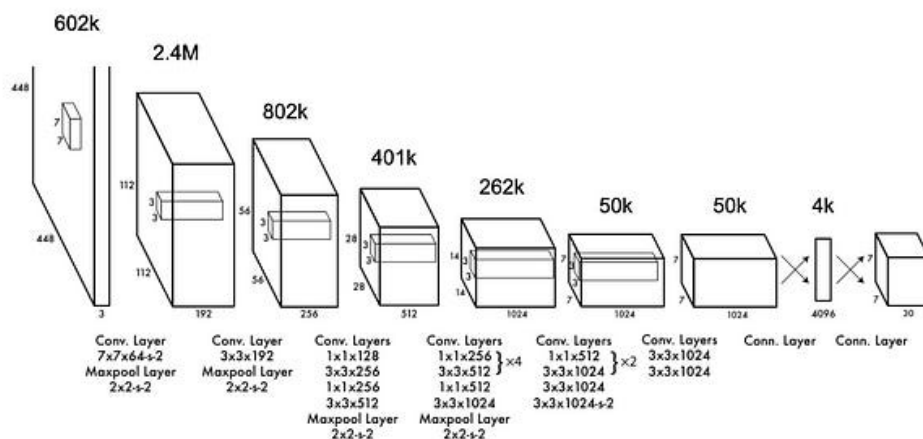
Rysunek 4: Augmentacja zdjęcia ciała (przed i po)



Rysunek 5: Augmentacja zdjęcia twarzy (przed i po)

3 Modele treningowe

Do wykonania zadania zastosowałam dwuetapowe podejście wykorzystujące architekturę YOLO (You Only Look Once) oraz ResNet. W pierwszym etapie wykorzystywałam uproszczoną implementację YOLO do detekcji twarzy myszy, a następnie model klasyfikacyjny oparty na ResNet-18 do określenia poziomu bólu. Zmodyfikowałam oryginalną implementację YOLO.



Rysunek 6: Oryginalna architektura YOLO

3.1 Implementacja YOLO

Implementacja modelu YOLO została uproszczona w stosunku do oryginalnej architektury, ale zachowuje kluczowe cechy umożliwiające detekcję obiektów w czasie rzeczywistym. Model składa się z dwóch głównych części:

3.1.1 Backbone (Szkielet)

Część odpowiedzialna za ekstrakcję cech z obrazu wejściowego składa się z pięciu bloków konwolucyjnych, z których każdy redukuje rozmiar obrazu o połowę przy jednoczesnym zwiększeniu liczby kanałów. Obraz wejściowy o rozmiarze 416×416 pikseli przechodzi przez następujące etapy:

- $416 \times 416 \rightarrow 208 \times 208$ (32 kanały)
- $208 \times 208 \rightarrow 104 \times 104$ (64 kanały)
- $104 \times 104 \rightarrow 52 \times 52$ (128 kanałów)
- $52 \times 52 \rightarrow 26 \times 26$ (256 kanałów)
- $26 \times 26 \rightarrow 13 \times 13$ (512 kanałów)

Każdy blok składa się z warstwy konwolucyjnej 3×3 , normalizacji batch oraz funkcji aktywacji LeakyReLU z parametrem 0.1, co zapobiega problemowi zanikającego gradientu. Końcowa warstwa konwolucyjna zwiększa liczbę kanałów do 1024, co zapewnia lepsze reprezentacje cech potrzebne do precyzyjniejszej detekcji.

3.1.2 Detection Head (Detekcja)

Głowa detekcyjna składa się z pojedynczej warstwy konwolucyjnej 1×1 , która przekształca 1024 kanały cech na tensor wyjściowy o wymiarach odpowiadających liczbie anchorów pomnożonej przez sumę współrzędnych bounding box (4), pewności obiektu (1) oraz liczby klas (1). Wynikowa siatka 13×13 dzieli obraz na 169 komórek, z których każda może zawierać do 3 potencjalnych detekcji.

3.1.3 Opis implementacji

Model SimpleYOLO został zaimplementowany w frameworku PyTorch. Składa się z sekwencyjnego modułu **backbone** zawierającego 5 bloków konwolucyjno-poolingowych, które stopniowo redukują rozmiar przestrzenny obrazu przy jednoczesnym zwiększaniu liczby filtrów. Ostatni blok konwolucyjny zwiększa liczbę kanałów do 1024. Głowa detekcyjna (**detection_head**) realizowana jest przez pojedynczą warstwę konwolucyjną 1x1, która mapuje wysokopoziomowe cechy na tensor detekcji o wymiarach [batch, anchors, grid, grid, 5+classes]. W funkcji **forward** tensor wyjściowy jest odpowiednio kształtowany do postaci [B, A, G, G, 5+C].

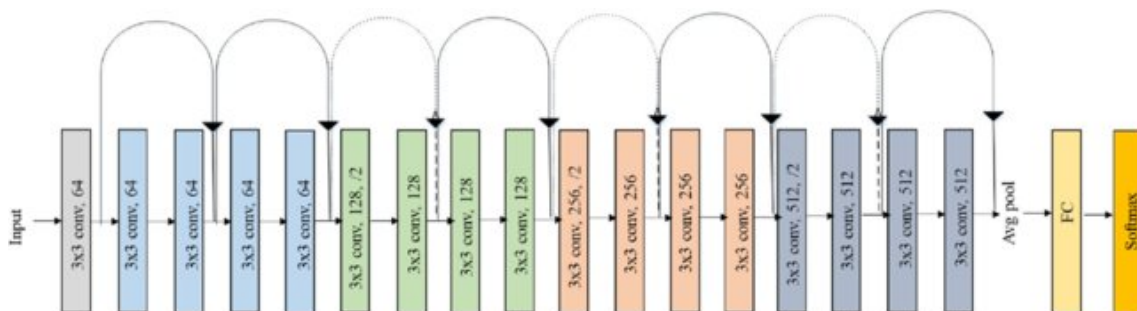
3.2 Klasyfikator bólu

3.3 Klasyfikator bólu

Do klasyfikacji poziomu bólu wykorzystano pretrenowaną sieć ResNet-18, która została dostosowana do zadania klasyfikacji trzech poziomów bólu ze skali MSG. Model został wytrenowany z wykorzystaniem techniki uczenia transferowanego (ang. transfer learning), gdzie pretrenowana sieć (na zbiorze ImageNet) została dostrojona do specyfiki klasyfikacji ekspresji bólu.

ResNet-18 charakteryzuje się głębokością 18 warstw i wykorzystuje połączenia rezydualne (ang. skip connections), które pozwalają na efektywne trenowanie głębokich sieci neuronowych bez problemu zanikającego gradientu. Architektura składa się z warstwy wejściowej (konwolucja 7x7), czterech bloków rezydualnych z progresywnym zwiększaniem liczby filtrów (64, 128, 256, 512) oraz warstwy klasyfikacyjnej z globalnym łączeniem (ang. global average pooling). Ostatnia warstwa w pełni połączona (ang. fully connected) została zmodyfikowana z oryginalnych 1000 klas na 3 klasy odpowiadające poziomom bólu.

Wykorzystanie połączeń rezydualnych umożliwia bezpośredni przepływ gradientów przez sieć poprzez dodawanie sygnału wejściowego do wyjścia każdego bloku rezydualnego ($F(x) + x$), co zapewnia stabilność trenowania i eliminuje problem zanikającego gradientu charakterystyczny dla głębokich sieci.



Rysunek 7: Architektura ResNet-18 z zaznaczonymi połączeniami rezydualnymi

3.3.1 Modyfikacja architektury

Ostatnia warstwa klasyfikacyjna (fully connected) została zastąpiona nową architekturą dostosowaną do specyfiki zadania:

- Warstwa Dropout (0.5) - zapobieganie overfittingowi
- Warstwa liniowa ($512 \rightarrow 128$ neuronów) - redukcja wymiarowości
- Funkcja aktywacji ReLU
- Druga warstwa Dropout (0.3) - dodatkowa regularyzacja
- Końcowa warstwa liniowa ($128 \rightarrow 3$ neurony) - klasyfikacja na 3 klasy

Taka architektura pozwala na wykorzystanie zaawansowanych reprezentacji wizualnych z ResNet-18 przy jednoczesnym dostosowaniu do specyficznego zadania rozpoznawania bólu u myszy.

3.3.2 Opis implementacji

Klasa PainClassifier dziedziczy po `nn.Module` z PyTorch. W metodzie `__init__` inicjalizowany jest pretrenowany model ResNet-18. Oryginalna warstwa `fc` (fully connected) jest zastępowana sekwencją nowych warstw: Dropout (0.5), Linear ($512 \rightarrow 128$), ReLU, Dropout (0.3) i końcowa warstwa liniowa ($128 \rightarrow 3$). W funkcji `forward` obraz jest przetwarzany przez zmodyfikowany model.

3.4 Funkcja straty YOLO

Zaimplementowano dedykowaną funkcję straty dla modelu YOLO, która jest kompozycją trzech składników odpowiadających różnym aspektom zadania detekcji obiektów.

3.4.1 Komponenty funkcji straty

Funkcja straty SimpleYOLOLoss składa się z następujących elementów:

1. Strata lokalizacji (Coordinate Loss) Wykorzystuje funkcję Mean Squared Error (MSE) do obliczania różnicy między przewidywanymi a rzeczywistymi współrzędnymi bounding boxów. Strata ta jest ważona parametrem $\lambda_{coord} = 5.0$, co nadaje większy priorytet precyzyjnej lokalizacji obiektów.

2. Strata pewności (Confidence Loss) Składa się z dwóch części:

- Strata pozytywna - dla komórek zawierających obiekty, używa Binary Cross Entropy
- Strata negatywna - dla komórek bez obiektów, ważona parametrem $\lambda_{noobj} = 0.5$

3. Strata klasyfikacji (Classification Loss) Wykorzystuje Binary Cross Entropy do przewidywania klas obiektów w przypadku gdy komórka zawiera obiekt.

3.4.2 Mechanizm anchorów

System wykorzystuje trzy predefiniowane anchora o rozmiarach $[10 \times 10]$, $[20 \times 20]$, $[30 \times 30]$ pikseli, dostosowane do typowych rozmiarów twarzy myszy w obrazach. Dla każdego obiektu wybierany jest anchor o najwyższym IoU (Intersection over Union), co poprawia jakość detekcji małych obiektów.

3.4.3 Opis implementacji

Klasa `SimpleYOLOLoss` dziedziczy po `nn.Module`. W konstruktorze inicjalizowane są anchor'y i parametry ważenia. Funkcja straty wykorzystuje MSE do strat lokalizacji i BCE-`WithLogitsLoss` dla strat pewności i klasyfikacji. W metodzie `forward` obliczane są poszczególne komponenty strat z uwzględnieniem masek obiektów i komórek tła.

3.5 System zarządzania modelem

Utworzono klasę `MousePainDetectionSystem`, która integruje oba modele i zapewnia końcowy interfejs do trenowania oraz predykcji. System realizuje kompleksowy pipeline przetwarzania obrazów myszy laboratoryjnych.

3.5.1 Architektura systemu

System składa się z następujących komponentów:

- **MousePainDataLoader** - moduł odpowiedzialny za wczytywanie i przetwarzanie danych ze zbioru
- **SimpleYOLO** - model detekcji twarzy myszy
- **PainClassifier** - model klasyfikacji poziomu bólu
- **Optymalizatory** - Adam optimizer z learning rate $1e-4$ dla obu modeli

3.5.2 Pipeline przetwarzania

Proces analizy obrazu przebiega w następujących etapach:

1. **Detekcja** - model YOLO lokalizuje twarze myszy na obrazie
2. **Ekstrakcja ROI** - wycięcie regionów zainteresowania zawierających twarze
3. **Preprocessing** - normalizacja i skalowanie do rozmiaru 224×224 dla ResNet
4. **Klasyfikacja** - predykcja poziomu bólu na skali 0-2
5. **Post-processing** - aplikacja funkcji softmax dla otrzymania prawdopodobieństw

3.5.3 Zarządzanie danymi

System automatycznie przygotowuje data loadery dla obu zadań:

- **Dataset detekcji** - obrazy 416×416 z oznaczonymi bounding boxami
- **Dataset klasyfikacji** - obrazy 224×224 z etykietami poziomów bólu
- **Batch processing** - przetwarzanie wsadowe z konfigurowalnymi rozmiarami batchy

3.5.4 Implementacja systemu

Klasa `MousePainDetectionSystem` w konstruktorze inicjalizuje:

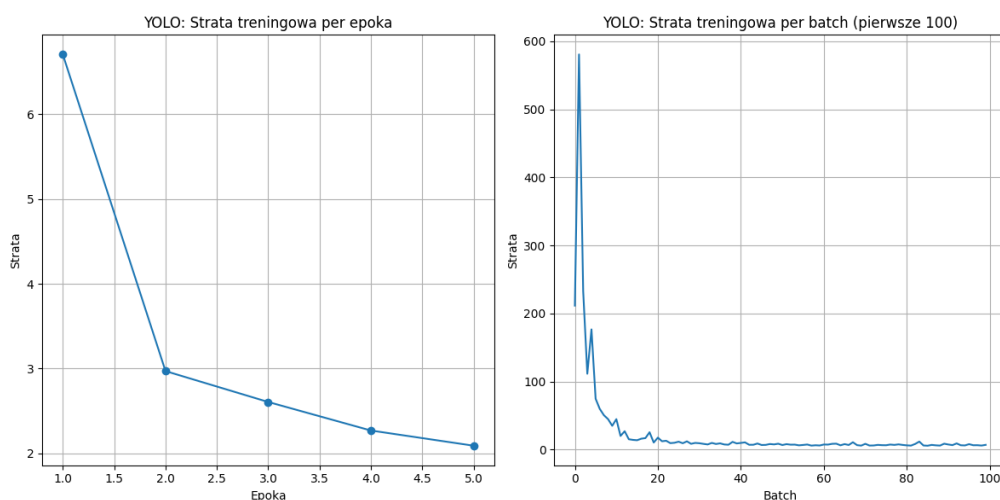
- Ładowarkę danych (`MousePainDataLoader`)
- Modele YOLO i klasyfikator bólu (przeniesione na odpowiednie urządzenie)
- Funkcje strat (`CrossEntropyLoss` dla klasyfikacji)
- Optymalizatory Adam dla obu modeli z $LR=1e-4$

Dodatkowo zawiera metody do trenowania, walidacji i predykcji, które zarządzają całym procesem uczenia, w tym propagacją wprzód, obliczaniem strat, propagacją wsteczną i aktualizacją wag.

3.6 Wyniki treningu

3.6.1 Wyniki YOLO

Trening modelu YOLO przebiegał stabilnie przez 5 epok. Funkcja straty wykazywała tendencję malejącą, co świadczy o prawidłowym procesie uczenia. Początkowa wartość średniej straty wynosiła około 8, a po zakończeniu treningu spadła do poziomu 2. Największy spadek straty odnotowano w pierwszych dwóch epokach, co jest typowe dla treningu sieci neuronowych.



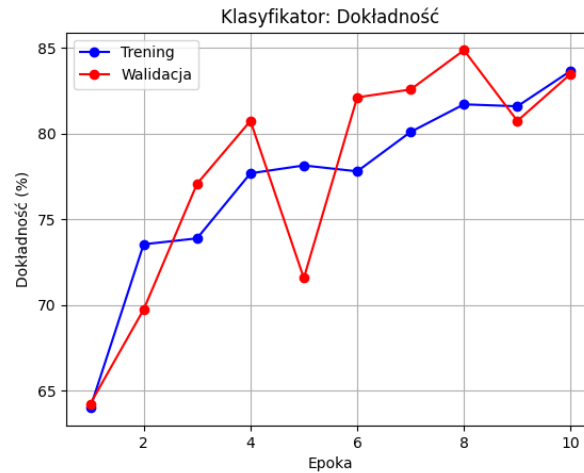
Rysunek 8: Funkcja straty podczas treningu modelu YOLO (5 epok)

Model YOLO osiągnął zadowalającą zdolność detekcji twarzy myszy, co potwierdzają wyniki wizualne przedstawione na rysunku 11. System prawidłowo lokalizuje regiony zainteresowania zawierające twarze myszy, co stanowi podstawę dla kolejnego etapu klasyfikacji.

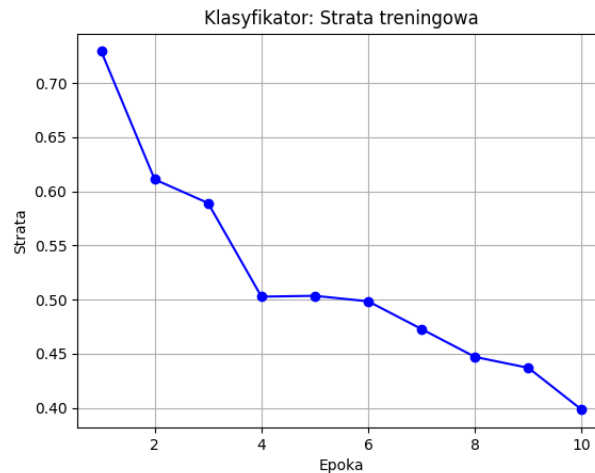
3.6.2 Wyniki klasyfikatora bólu

Klasyfikator ResNet-18 został wytrenowany na zbiorze zawierającym obrazy twarzy myszy z etykietami bólu w skali 0-2 (MGS). Proces treningu przebiegał przez 10 epok z podziałem danych na zbiór treningowy (80%) i walidacyjny (20%).

Najlepszą dokładność walidacyjną osiągnięto w 9. epoce. Dokładność treningowa wzrosła systematycznie, osiągając 84% po 10 epokach. Różnica między dokładnością treningową a walidacyjną wskazuje na umiarkowane przeuczenie modelu, co jest częstym zjawiskiem przy ograniczonym zbiorze danych.



Rysunek 9: Dokładność klasyfikatora podczas treningu i walidacji



Rysunek 10: Funkcja straty klasyfikatora podczas treningu

3.7 Analiza wyników

Dwuetapowe podejście z wykorzystaniem YOLO i ResNet-18 okazało się skuteczne w automatycznej ocenie bólu u myszy laboratoryjnych. Model YOLO zapewnił precyzyjną lokalizację twarzy myszy, eliminując konieczność manualnego wyznaczania regionów zainteresowania. Klasyfikator ResNet-18 osiągnął dokładność porównywalną z wynikami publikowanymi w literaturze dla podobnych zadań.

Ograniczeniem systemu jest relatywnie niewielki zbiór danych treningowych (3309 obrazów), co może wpływać na generalizacyjne zdolności modelu. Zastosowanie augmentacji

danych (odbicia poziome, skalowanie, zmiany kolorów HSV) pomogło w zwiększeniu różnorodności zbioru treningowego i poprawie stabilności treningu.

3.7.1 Przykłady detekcji



Rysunek 11: Przykładowa detekcja (z grymasem)

Rysunek 11 przedstawia przykładowe wyniki działania systemu na obrazach testowych. System prawidłowo identyfikuje twarze myszy i klasyfikuje poziom bólu, co potwierdza skuteczność zaproponowanego podejścia.

4 Bibliografia

1. Langford, D.J., Bailey, A.L., Chanda, M.L., Clarke, S.E., Drummond, T.E., Echols, S., Glick, S., Ingrao, J., Klassen-Ross, T., Lacroix-Fralish, M.L., Matsumiya, L., Sorge, R.E., Sotocinal, S.G., Tabaka, J.M., Wong, D., van den Maagdenberg, A.M., Ferrari, M.D., Craig, K.D., Mogil, J.S. (2010). Coding of facial expressions of pain in the laboratory mouse. *Nature Methods*, 7(6), 447-449.
2. Vidal, A., Jha, S., Hassler, S., Price, T., Busso, C. (2022). Face detection and Grimace Scale Prediction of White Furred Mice. *Machine Learning for Workflow Applications (MLWA)*.
3. Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788.

4. He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.
5. Multimodal Speech Processing Laboratory, The University of Texas at Dallas. White Furred Mice Dataset. <https://lab-msp.com/WhiteFurredMiceDataset/>